

Templates:

<https://static.remotasks.com/uploads/68598fb6f5de06fb594918bd/hawkins.zip>

Small guide: running the test-output parser (parsing.py

) on your test logs

Your parsing.py script is designed to take **two input “documents”**:

- a file containing the **test run stdout**
- a file containing the **test run stderr**

...and produce a **JSON file** with structured results:

```
None
{
  "tests": [
    {
      "name": "test_name", "status":
      "PASSED|FAILED|SKIPPED|ERROR"
    }
  ]
}
```

1) Generate the input files (stdout + stderr)

Option A — using

run.sh

run.sh is meant to run your full test suite, but right now it's a template (it prints “Running all tests...” and doesn't run anything yet). You'll need to edit the run_all_tests() function to actually execute your tests.

Once run.sh runs real tests, capture logs like this:

```
None  
chmod +x run.sh  
./run.sh > stdout.txt 2> stderr.txt
```

Option B — run tests directly

If you already have a test command, just redirect outputs:

```
None  
<your-test-command> > stdout.txt 2> stderr.txt
```

(Example: `pytest -q > stdout.txt 2> stderr.txt`)

2) Run the parser

Run parsing.py by providing the two log files and an output path:

```
None  
python3 parsing.py stdout.txt stderr.txt results.json
```

After it runs, you should have results.json containing the extracted test list.

3) Important note:

parse_test_output()

is currently unimplemented

Right now, parsing.py will raise:

None

```
NotImplementedError('Implement the test output parsing logic')
```

So the parser **won't work until you implement** this function:

None

```
def parse_test_output(stdout_content: str, stderr_content: str)
-> List[TestResult]:
    ...
```

What you implement depends on your test framework (pytest, junit output, go test, cargo test, etc.). The goal is to return a list of:

- `TestResult(name="<test name>", status=TestStatus.PASSED/FAILED/SKIPPED/ERROR)`

Once you implement it, the command in step (2) will produce the JSON automatically.

4) Running inside Docker (typical workflow)

Your Dockerfile is also a template: you must set a real base image and repo clone URL.

Typical flow (once Dockerfile is filled in):

None

```
docker build -t test-parser .
docker run --rm -it test-parser
```

Inside the container, you'd do:

None

```
./run.sh > stdout.txt 2> stderr.txt
python3 parsing.py stdout.txt stderr.txt results.json
```

If you want the output on your host machine, mount a volume:

None

```
docker run --rm -it -v "$PWD:/app" test-parser bash -lc \  
  "./run.sh > stdout.txt 2> stderr.txt && python3 parsing.py \  
  stdout.txt stderr.txt results.json"
```

This writes stdout.txt, stderr.txt, and results.json into your current folder.

Quick checklist

- run.sh actually runs your tests (not just the placeholder echo)
- you generated stdout.txt and stderr.txt
- you implemented parse_test_output() in parsing.py
- you ran: python3 parsing.py stdout.txt stderr.txt results.json
- results.json contains the expected { "tests": [...] } structure